

LM in Informatica

Distributed Software Systems
2026-27

Paolo Ciancarini

University of Bologna, Italy

Who am I

Professor of Computer Science@UniBo

Researcher on sw technologies for distributed
programming and genAI

... and who are you?

Prerequisites

- This course presents the fundamental ideas on **distributed systems** and their principles of construction
- We also present middleware technologies and the related methods for distributed programming
- We will present some basic ideas on DS and genAI
- Prerequisites:
 - Be a graduate student – this is an advanced course
 - Be able to design program (object oriented and patterns)
 - Basic notions of networking and operating systems
 - Basic notions of software engineering and architecture

Effort and scheduling

- Effort : 6 cfu
- Course duration: from sept to december
- Class hours:
 - Tuesday 13-15 room Ercolani 3
 - Wednesday 13-15 room Ercolani 3

Remark: Some lectures will be online

Syllabus

1. Foundations of distributed systems

2. Communication & coordination (eg. RPC/gRPC, messaging, service mesh)

3. Consistency, consensus & fault tolerance

(eg. Raft/Paxos, CRDTs, CAP/PACELC)

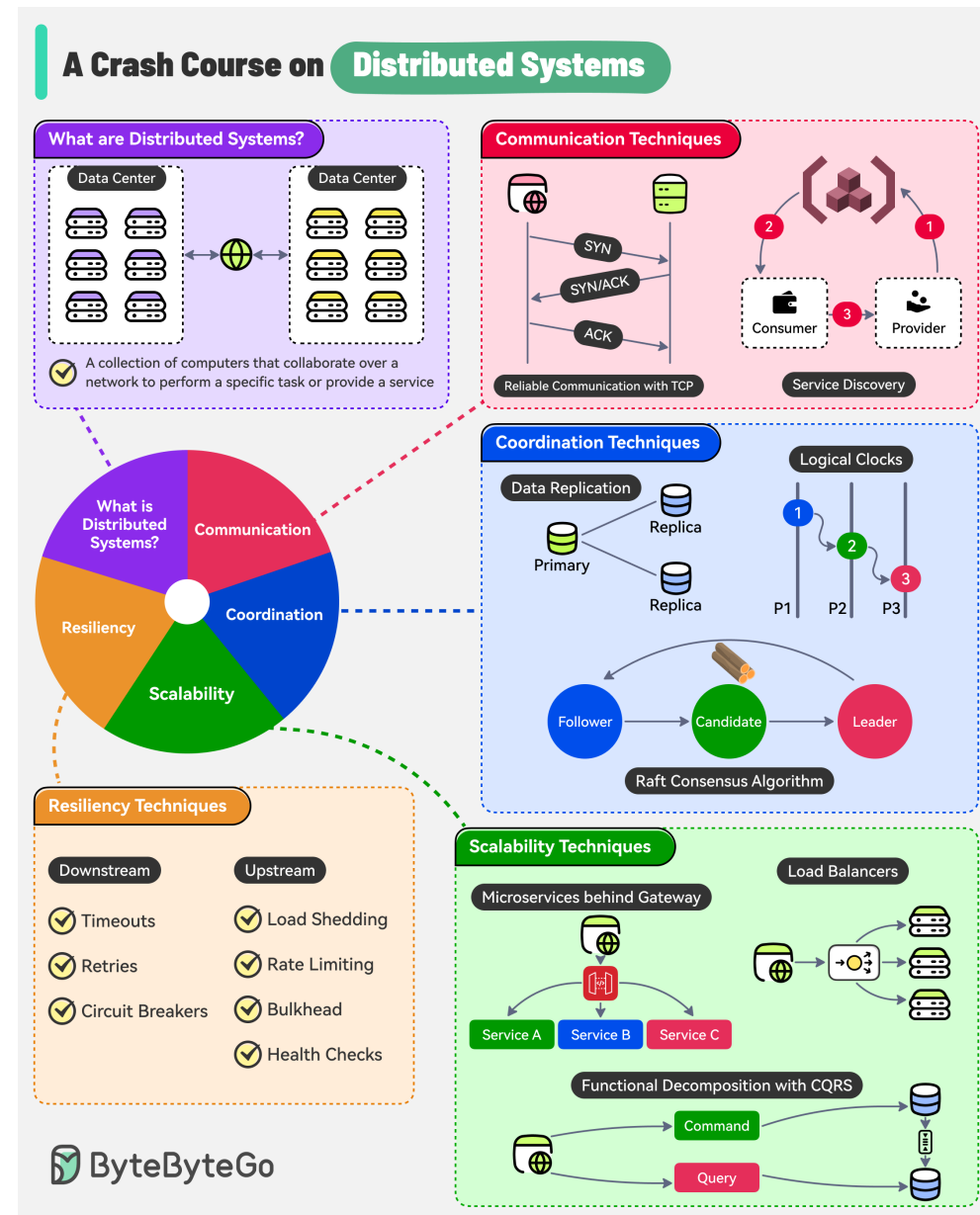
4. Platforms & orchestration

(eg. Kubernetes, serverless, observability)

5. Distributed systems for AI

(eg. model serving, distributed training, multi-agent systems)

Hands-on work on specific problems in distributed programming



Goals of this course

- Understand principles and concepts underlying **distributed computing**: communication, concurrency, coordination and related algorithms
- Learn how to **design** simple reliable **distributed applications** using (open source) **middleware**
- Gain **practical** experience on the development of simple **distributed systems** (eg. in **Java** or **Python**)
- Explore how GenAI is reshaping distributed systems, from **distributed model serving/training** to **multi-agent orchestration**

Exam

- Midterm report & presentation: 30%
- Project (personal or in team): 50%
- Attendance and labs: +/- bonus up to 10%
(negative if you do not attend classes or no labs)

Mid-term (individual) presentation

Your task:

- you have to deliver a small paper (five pages) and a presentation (max 10 slides, talk 15')

Possible topics (examples):

- Middleware for distributed applications
- Important examples of distributed systems (architecture, behavior)
- Orchestrating distributed applications with GenAI

The midterm

Each student chooses a *different* topic: eg. a real system or a middleware platform

Look for 2-3 references to support your choice
(scientific papers or books –use Google Scholar)

Write your choice in the shared googledoc I will
publish on the Telegram channel and on Virtuale

Prepare 10 slides

Upload your slides in the moodle Virtuale, pdf format

Present the slides to class

Suggested ToC

1. System purpose and application context
2. Actors, components, and deployment environment
3. Communication model and interfaces
4. Distribution transparencies and their limitations
5. Replication, consistency, and failure handling
6. Trust boundaries, threats, and mitigations
7. Failure scenario, recovery, and observability
8. Conclusions and references

Suggested choices

Topic area	Suggested choices
RPC and APIs	gRPC, Apache Thrift, GraphQL, OpenAPI
Messaging and event streaming	Kafka, Pulsar, RabbitMQ, NATS, RocketMQ, MQTT
Coordination and workflows	etcd, Consul, ZooKeeper, Temporal
Distributed computing and processing	Ray, Dask, Apache Flink, Apache Spark
Distributed data and storage	Cassandra, CockroachDB, Redis, FoundationDB, Ceph
Cloud and serverless middleware	Kubernetes, Dapr, Knative, Apache OpenWhisk
GenAI agent orchestration	LangGraph, CrewAI, OpenAI Agents SDK, Constructor Fabric
Observability	OpenTelemetry, Prometheus, Jaeger, Grafana Tempo

Grading the midterm

Architecture and distributed concepts: 30%

Failure and security analysis: 25%

Technical depth and evidence: 20%

Scenario and diagrams: 15%

Presentation quality: 10%

Important: AI assistance is permitted but must be disclosed. Students remain responsible for claims, diagrams, and references. Fabricated or unverified references invalidate the submission.

Timeline

Choose topic by oct 30 (on googlesheet I will prepare)

Link the references you want to use

I can help with additional material

Upload slide pack: Nov 15, 2026

Presentations online: December 2-9, 2026

Labs

Exercises

I will assign weekly during the course

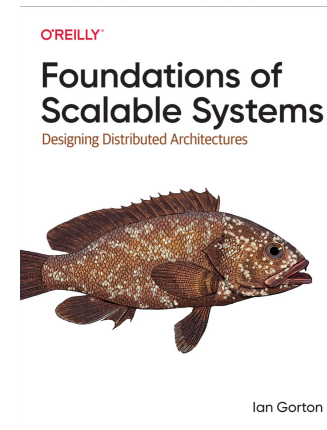
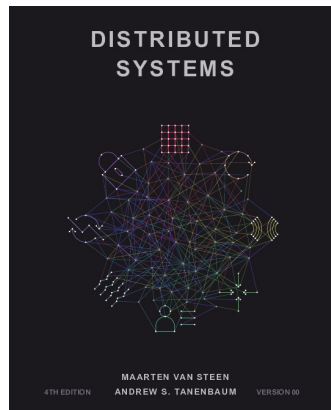
Solutions (by individuals or pairs)

are expected after one week max, presented and discussed in class

Channels

- **Telegram** (course): [UniBo Distributed Sw Systems](#)
- **Telegram** @PaLoCaPa (personal)
- **Email** `snmp://paolo.ciancarini@unibo.it/`
- **Linkedin**: search me and ask for contact!

Textbooks



[vanSteen and Tanenbaum, *Distributed Systems*, 4ed, 2024](#)

Gorton, *Foundations of scalable systems*, O'Reilly, 2022

Additional material (slides, papers, books) distributed during the course

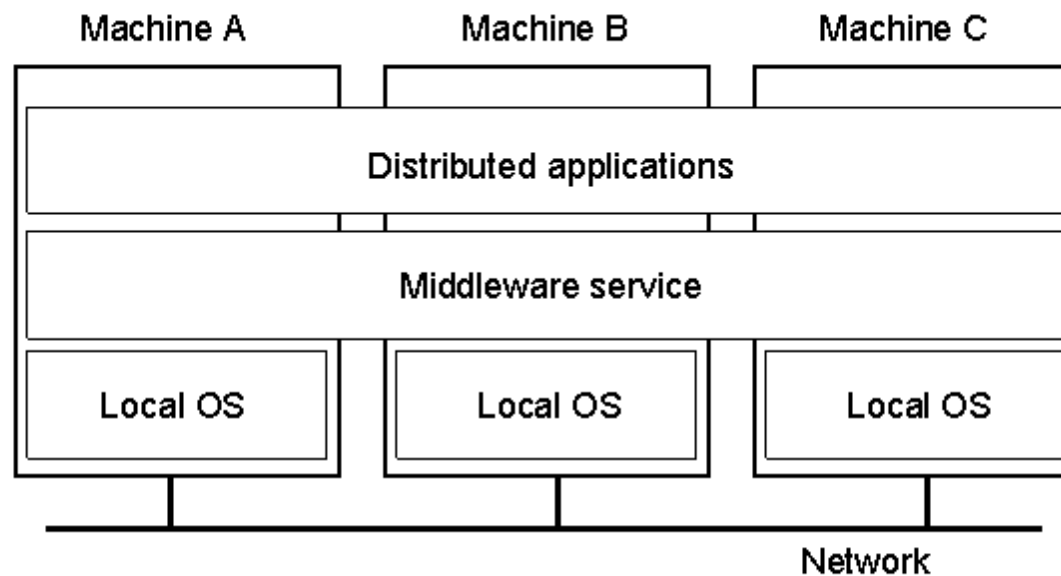
E.g. <https://www.distributedsystemscourse.com>

Questions?



Definition: distributed system

A distributed system is a collection of independent computers that appears to its users as **a single coherent system** thanks to some **middleware** which makes transparent the underlying infrastructure



Lamport's definition of a Distributed System

*A distributed system is one in which
the failure of a computer
you didn't even know existed
can render your own computer unusable*

(Leslie Lamport)

Distributed vs centralized state

Centralized system: state stored on a single computer

- simpler to design
- easier to understand
- global clock



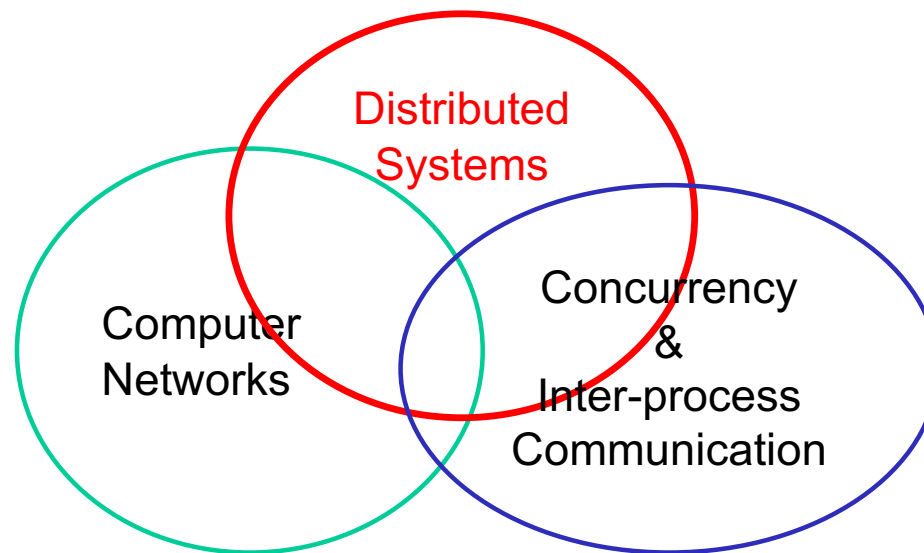
Distributed system: state divided over multiple computers

- Scalable
- Fault tolerant (under specific assumptions)
- Complex: no global time and multiple possible failures

Areas

Well designed distributed systems hide network problems ...
but in order to understand and develop them, we need
competencies in

- Computer networks and Internet
- Concurrency & Inter-process communication



Two consequences of distribution

- Data travel at (max) the speed of light, but usually the speed is quite slower: latency in communication
- Independent things fail independently

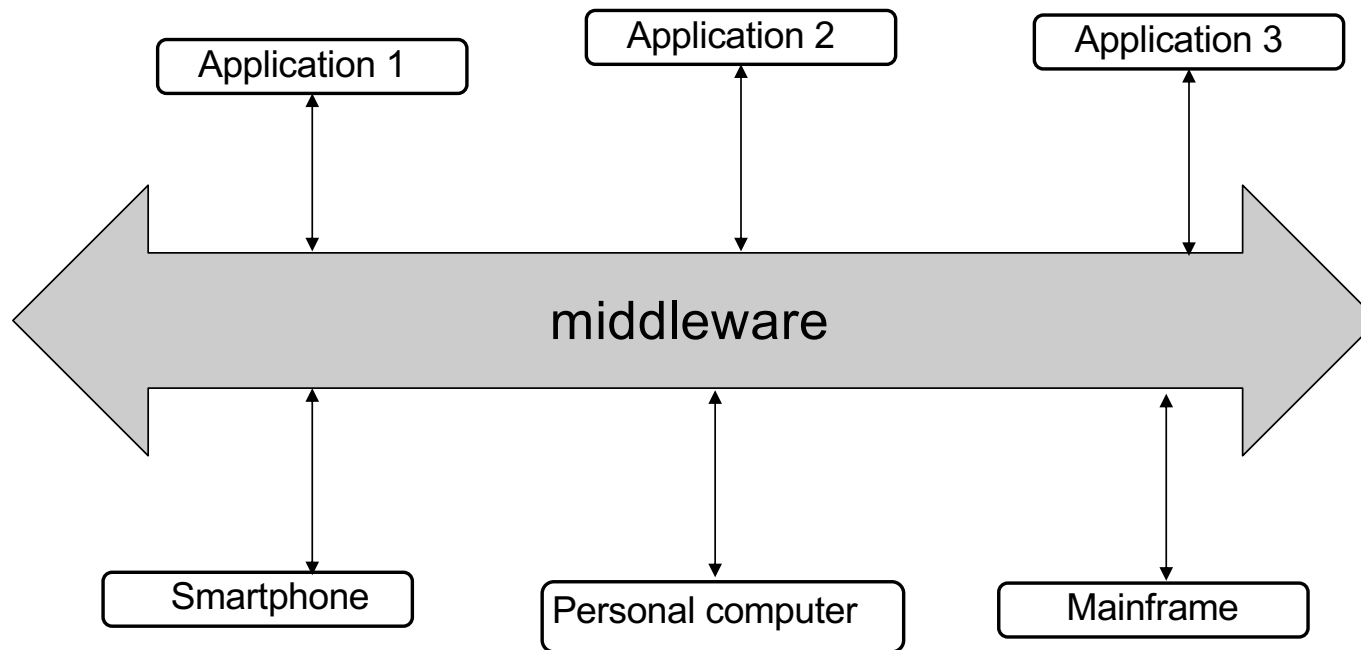
Building distributed systems means dealing with space, time, and having more than one active computing element able to change state.

These constraints allow several possible system designs, and after this course you should have a better idea of how distance, time, and consistency models interact.

Languages for distributed computing

- There are problems in distributed system environments that do not exist in single-machine environments.
- Latency, concurrency, and (partial) failures are three problems that make distributed (remote) computing fundamentally different from centralized (local) computing
- Languages for distributed programming are often normal languages (eg C++, Java, Python) extended with some **middleware** able to deal with communication, coordination, and consistency

Middleware



Middleware as an infrastructure for distributed systems

Variants of distributed systems

Internet of Things

Cloud computing

Fog computing

Edge computing

Service oriented computing

Multi-agent systems

Distributed Systems for AI: why it matters

Modern AI systems are inherently distributed: training, serving, agent orchestration span many machines

The same principles (consistency, latency, fault tolerance) apply, but at unprecedented scale

New failure modes: accelerator failures, stragglers, non-determinism in numerical computation

This block links classic distributed-systems theory to modern ML infrastructure and multi-agent AI

Model serving at scale

Serving large models requires sharding across multiple GPUs/nodes (tensor and pipeline parallelism)

Load balancing, caching (KV-cache) and batching are classic distributed-systems problems

Frameworks: vLLM, NVIDIA Triton Inference Server, Ray Serve

Trade-offs: latency vs throughput, cost vs availability

Distributed training

Data, model, and hybrid parallelism strategies for training large models

Synchronization: parameter servers, all-reduce, gradient compression

Fault tolerance: checkpointing, elastic training on preemptible resources

Frameworks: PyTorch Distributed, Ray Train, DeepSpeed, Horovod

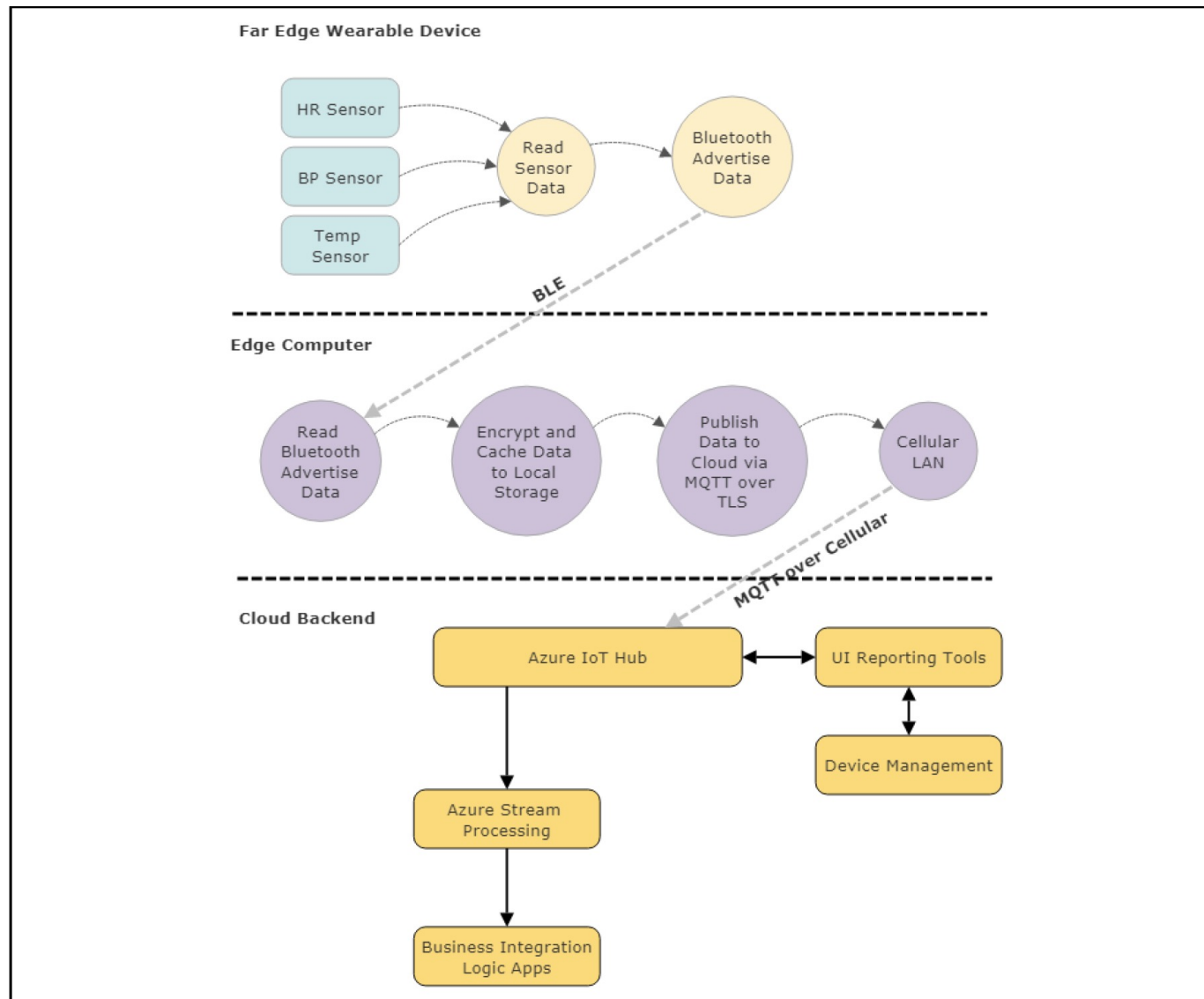
Multi-agent systems and LLM orchestration

LLM-based agents as autonomous distributed components communicating via messages and tools

Coordination patterns: orchestrator-worker, blackboard, negotiation protocols

Consistency challenges: shared state, memory, tool-call ordering across agents

Connects to game-theoretic reasoning — multi-agent systems as a testbed, as in chess-playing AI



From Lea, *IoT and Edge computing for architects*, Packt 2020

Centralized vs distributed systems

Criteria	Centralized	Distributed
Economics	Low (expensive)	High (cheap)
Availability	low	high
Complexity	low	high
Consistency	simple	difficult
Scalability	poor	good
Technology	homogeneous	heterogeneous
Security	high	low

Which problems should we solve when programming a distributed system?

Access (eg. How do we name resources?)

Transparency (we would like to ignore distribution issues)

Fault tolerance (we have multiple sources to get a service)

Latency (data speed is not infinite!)

... programming without shared memory and no absolute (unique) time is DIFFICULT

Transparency

- A transparency is some aspect of a distributed system that is hidden – abstracted - from the user (programmer, system developer, user or application program), so it can be ignored
- A transparency is implemented by some set of mechanisms in the distributed system at a layer below the interface where the transparency is required
- A number of basic transparencies have been defined; not all of these are appropriate for every distributed system, or are available at the same level of interface.
- In fact, all transparencies have an associated cost, and it is extremely important for implementor to be aware of this cost

There are several transparencies

Transparency: access

Access transparency There should be **no difference between local and remote access methods**. In other words, the cost of explicit communication may be hidden.

For instance, from a user's point of view, access to a remote service such as a printer should be identical with access to a local printer.

From a programmers point of view, the access method to a remote object may be identical to access a local object of the same class.

Moreover, since the semantics of remote access are more complex, particularly in case of failure, this means that a local access should be a special case of remote access.

Remote access will not always look like local access in that certain facilities may not be reasonable to support (for example, global exhaustive searching of a distributed system for a single object may be unreasonable in terms of network traffic).

Transparency: concurrency

Concurrency transparency Users and applications should be able to access shared data or objects **without interference between each other**.

This requires very complex mechanisms in a distributed system, since there exists true concurrency rather than the simulated concurrency typical of a centralized system.

For example, a distributed printing service must provide the same atomic access per file as a central system so that printout is not randomly interleaved.

Transparency: replication and fault tolerance

Replication Transparency If the system provides replication for availability or performance reasons, it should not concern the user (as for all transparencies, we include the applications programmer as a user)

Fault Transparency If software or hardware failures occur, these should be hidden from the user. For instance, in a networked system, it is often hard to tell the difference between a failed and a slow running process or processor. Fault transparency can be difficult since partial failure of the communication subsystem is possible, and this may not be reported. As far as possible, fault transparency will be provided by mechanisms that relate to access transparency. However, when the faults are inherent in the distributed nature of the system, then access transparency may not be maintained. The mechanisms that allow a system to hide faults may result in changes to access mechanisms (e.g. access to reliable objects may be different from access to simple objects).

Transparencies: location and migration

Location transparency The details of the topology of the system should be of no concern to the user. The location of an object in the system may not be visible to the user or programmer. (This differs from access transparency in that both the naming and access methods may be the same). Names may give no hint as to location.

Migration transparency If objects (processes or data) migrate (to provide better performance, or reliability, or to hide differences between hosts), this change of location should be hidden from the user.

Transparency: performance, scaling

Performance Transparency The configuration of the system should not be apparent to the user in terms of performance. This may require complex resource management mechanisms. It may not be possible at all in cases where resources are only accessible via low performance networks.

Scaling Transparency A system should be able to grow without affecting application algorithms. Graceful growth and evolution is an important requirement for most enterprises. A system should also be capable of scaling down to small environments where required, and be space and/or time efficient as required.

Transparencies in a Distributed System

Transparency	Description
Access	Hide differences in data representation and how a resource is accessed
Location	Hide where a resource is located
Migration	Hide that a resource may move to another location
Relocation	Hide that a resource may be moved to another location while in use
Replication	Hide that a resource may be shared by several competitive users
Concurrency	Hide that a resource may be shared by several competitive users
Failure	Hide the failure and recovery of a resource
Persistence	Hide whether a (software) resource is in memory or on disk

Hands on Lab 1

Location transparency in distributed file storage

1. Goal

Understand how location transparency allows clients to access files without knowing which server stores them, and identify its limits.

2. Scenario

Your team is designing a file storage system distributed across three servers. Users access files through logical paths such as `/shared/reports/results.pdf`, without specifying a server name or address. Design the mechanism that maps these paths to the servers storing the files. This is a design exercise; implementation is not required.

3. Tasks

- Propose a *naming scheme* and explain where the mapping from logical paths to storage locations is maintained.
- Trace a *read* request: how does the client locate a file and retrieve its contents?
- Suppose a file moves to another server. Explain how the mapping is updated and how you handle clients with outdated location information.
- Discuss what happens when the mapping service or a storage server becomes *unavailable*. Distinguish hiding location from hiding failures.
- Identify where *access permissions* are checked: hiding a file's location must not bypass authorization.

4. Deliverable

Prepare a short presentation (maximum four slides) containing an architecture diagram, a walkthrough of a read request, your handling of relocation and failures, and one design trade-off.

Questions?

